

# Training a Transformer from Scratch

A modern decoder-only GPT (RoPE, RMSNorm, SwiGLU)

Marcos Lahoz

## Abstract

We implement a decoder-only Transformer language model from scratch in PyTorch — using only tensor primitives, no pretrained or library model classes — with the components found in current LLMs: pre-norm **RMSNorm**, **rotary position embeddings** (RoPE), **SwiGLU** feed-forward blocks, weight tying and a **KV cache** for generation. We verify the implementation on a synthetic copy task (98.4% exact-copy accuracy), train it as a word-level language model on TinyStories. The code is a compact, reproducible reference for how a modern GPT is built and trained.

## 1 Architecture

The model is a stack of  $L$  pre-norm Transformer blocks. For hidden states  $x$ , each block computes

$$x \leftarrow x + \text{Attn}(\text{RMSNorm}(x)), \quad x \leftarrow x + \text{SwiGLU}(\text{RMSNorm}(x)).$$

**RMSNorm** [3] normalises by the root-mean-square of the activations (no mean subtraction, no bias). **RoPE** [2] rotates the query/key vectors by position-dependent angles, injecting relative position directly into the attention dot-product. Attention is the standard causal multi-head self-attention; for generation we keep a per-layer **KV cache** so each new token costs  $O(T)$  rather than  $O(T^2)$ . The MLP is **SwiGLU** [4],  $\text{SwiGLU}(x) = W_{\text{down}}(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x)$ . The token-embedding and output projection share weights (weight tying).

## 2 Implementation validation

Before training on text we validate the full pipeline (forward, loss, backward, and cached generation) on a *copy task*: the input is a random length-8 sequence, a separator, and the same sequence again; only the copied part is scored. Solving it requires attending back to the prefix. A 2-layer, 64-dim model (107k parameters) reaches a loss of  $2 \times 10^{-4}$  in 300 steps and **98.4%** exact-copy accuracy when generating with the KV cache — confirming attention, RoPE, masking and sampling are correct. Reproduce with `python scripts/smoke_test.py`.

## 3 Language modeling

We BPE-tokenise (GPT-2 vocabulary) a TinyStories stream into a flat token array and sample fixed-length blocks. Training uses AdamW with decoupled weight decay, a cosine learning-rate schedule with warmup, gradient clipping, gradient accumulation and mixed precision (bf16/fp16). Validation loss and perplexity are logged periodically; `scripts/generate.py` samples text from a checkpoint.

A 25.3M-parameter model (8 layers, 8 heads,  $d=512$ , context 256) trained on 44.7M TinyStories tokens for 5,000 iterations ( $\sim 11$  min on a single GPU) reaches a validation loss of **1.54** (perplexity **4.67**), down from 10.92 at initialisation, and generates coherent short stories, e.g. for the prompt “*Once upon a time*” it continues with a complete, on-topic narrative about two friends playing in a park.

## 4 Discussion

The project is a clean, dependency-light reference implementation of a modern GPT that is both *correct* (validated end-to-end) and *scalable*. Natural extensions: grouped-query attention, FlashAttention / `scaled_dot_product_attention`,  $\mu$ P or careful LR scaling, and longer context.

## References

- [1] Vaswani et al. (2017). *Attention Is All You Need*. NeurIPS.
- [2] Su et al. (2021). *RoFormer: Enhanced Transformer with Rotary Position Embedding*.
- [3] Zhang & Sennrich (2019). *Root Mean Square Layer Normalization*. NeurIPS.
- [4] Shazeer (2020). *GLU Variants Improve Transformer*.